

TECHNICAL TRAINING COURSE

Parameter and Return Types

JAX-RS 2.x

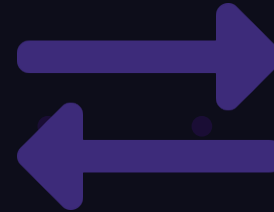
- Simple types · @Consumes / @Produces
- @XXXParam annotations · @DefaultValue
- Return types · Binary content · File delivery

Java 11 LTS

Tomcat 9.0

JAX-RS 2.1

Jersey 2.x



COURSE AGENDA

7 Sections | Input/Output Data Handling in JAX-RS

01

Simple Parameter Types

05

Return Types

02

@Consumes and @Produces

06

Binary Content

03

@XXXParam Annotations

07

Delivering a File

04

The @DefaultValue Annotation

Simple Parameter Types — Auto-Conversion Rules

JAX-RS automatically converts the HTTP string into the declared Java type:

1

Primitive

int, long, boolean, double

2

Boxed type

Integer, Long, Boolean, Double

3

String

No conversion needed

4

fromString(String)

UUID, custom types

5

valueOf(String)

Enum, BigDecimal

6

new T(String)

Types with String constructor

7

List/Set/SortedSet

Of any of the above

Conversion failure: @PathParam → 404 | @QueryParam/@HeaderParam/@FormParam → 400

@Consumes and @Produces

Content negotiation — the runtime matches media types on both sides of the conversation

@Produces — what we SEND

Matched against Accept header

415 if no match → wrong Accept

Multiple types listed in preference order

`*/*` = wildcard fallback

First listed = default when `*/*` sent

@Consumes — what we ACCEPT

Matched against Content-Type header

415 Unsupported Media Type if no match

Only applies when there is a request body

POST / PUT / PATCH typically use this

GET / DELETE ignore @Consumes

Content Negotiation — Class vs Method Level

```
@Path("/products")
@Produces(APPLICATION_XML)      // class default
@Consumes(APPLICATION_XML)     // class default
public class ProductResource {

    @GET                         // inherits @Produces(JSON)
    public List<Product> list() { ... }

    @GET @Path("{id}")
    @Produces({APPLICATION_XML,  // overrides class default
              APPLICATION_XML})
    public Product get(@PathParam("id") String id) { ... }
    // Accept: application/xml → XML response
    // Accept: text/html      → 406 Not Acceptable

    @GET @Path("export")
    @Produces("text/csv")       // overrides with CSV
    public Response exportCsv() {
        return Response.ok(buildCsv())
            .header("Content-Disposition",
                    "attachment; filename=products.csv")
            .build();
    }
}
```

@XXXParam Annotations — All Sources

@PathParam

URI template variable {name}

/products/{id}

@QueryParam

URL query string ?name=value

?page=0&size=20

@HeaderParam

HTTP request header

Authorization: Bearer ...

@FormParam

Form body (urlencoded / multipart)

username=alice&pw=secret

@CookieParam

Cookie header sent by client

Cookie: SESSION=xyz

@MatrixParam

Semicolon params in path segment

/products;cat=books

@BeanParam

Aggregates multiple @Param into a bean

ProductFilter bean

@Context

JAX-RS framework objects

UriInfo, HttpHeaders, ...

@BeanParam — Cleaner Method Signatures

```
// Without @BeanParam — messy
@GET
public List<Product> list(
    @QueryParam("category") String cat,
    @QueryParam("minPrice") BigDecimal min,
    @QueryParam("maxPrice") BigDecimal max,
    @QueryParam("page") @DefaultValue("0") int pg,
    @QueryParam("size") @DefaultValue("20") int sz,
    @QueryParam("sort") @DefaultValue("name") String sort,
    @HeaderParam("Accept-Language") String lang) {
    // ... 7-parameter chaos
}
```

```
// ProductFilter bean
public class ProductFilter {
    @QueryParam("category")
    public String category;
    @QueryParam("minPrice")
    public BigDecimal minPrice;
    @QueryParam("page")
    @DefaultValue("0")
    public int page;
    @QueryParam("size")
    @DefaultValue("20")
    public int size;
    @HeaderParam("Accept-Language")
    public String language;
}

// With @BeanParam — clean!
@GET
public List<Product> list(
    @BeanParam ProductFilter f) {
    return repo.find(f);
}
```

@DefaultValue — Self-Documenting Defaults

```
// Without @DefaultValue — manual null checks everywhere
@GET public List<Product> list(@QueryParam("page") int page,
    @QueryParam("size") Integer size, @QueryParam("sort") String sort) {
    int s = (size != null) ? size : 20; String o = (sort != null) ? sort : "name";
    return repo.find(page, s, o); }

// With @DefaultValue — clean, self-documenting contract
@GET public List<Product> list(
    @QueryParam("page") @DefaultValue("0") int page,
    @QueryParam("size") @DefaultValue("20") int size,
    @QueryParam("sort") @DefaultValue("name") String sort,
    @QueryParam("active") @DefaultValue("true") boolean active) {
    return repo.find(page, size, sort, active); }
```

Default fires ONLY when parameter is completely absent

Empty string ?page= injects null or empty, NOT the default

The default string is parsed by the same rules as a real value

Works with all @Param annotations (not just @QueryParam)

Return Types

Return Type	Default Status	Notes
<code>void</code>	<code>204 No Content</code>	No body sent
<code>POJO / DTO</code>	<code>200 OK + serialised body</code>	JAXB marshals to XML automatically (@XmlRootElement)
<code>Collection<T></code>	<code>200 OK + JSON array</code>	Use GenericEntity to preserve List<T> type for JAXB
<code>Response</code>	<code>As set by builder</code>	Full control: status, headers, body
<code>Response (null)</code>	<code>200 OK</code>	Empty body but headers still available
<code>CompletionStage<T></code>	<code>Async - later</code>	Non-blocking; JAX-RS 2.1+

Building Responses — ResponseBuilder API

```
// 201 Created + Location
@POST
public Response create(Product p,
    @Context UriInfo ui) {
    Product saved = repo.save(p);
    URI loc = ui.getAbsolutePathBuilder()
        .path(saved.getId()).build();
    return Response.created(loc)
        .entity(saved).build();
}

// 204 No Content
@DELETE @Path("{id}")
public Response delete(
    @PathParam("id") String id) {
    if (!repo.delete(id))
        return Response.status(404).build();
    return Response.noContent().build();
}
```

Key Builder Methods

<code>Response.ok(entity)</code>	200 with body
<code>Response.created(uri)</code>	201 + Location
<code>Response.noContent()</code>	204
<code>Response.notModified()</code>	304
<code>Response.status(code)</code>	Any status
<code>.entity(object)</code>	Set body
<code>.header(name,value)</code>	Add header
<code>.cookie(NewCookie)</code>	Set-Cookie
<code>.tag(EntityTag)</code>	ETag header
<code>.build()</code>	Finalise

Binary Content — Reading and Writing

```
// READ: byte[] — full body in memory
@POST @Path("/images/{id}")
@Consumes("image/*")
public Response upload(
    @PathParam("id") String id,
    byte[] data) {
    imageService.store(id, data);
    return Response.noContent().build();
}

// READ: InputStream — streaming
@POST @Path("/upload")
@Consumes(APPLICATION_OCTET_STREAM)
public Response uploadStream(
    @PathParam("id") String id,
    InputStream stream) {
    storage.save(id, stream); // constant memory
    return Response.noContent().build();
}
```

```
// WRITE: StreamingOutput (lazy)
@GET @Path("/reports/{id}/pdf")
@Produces("application/pdf")
public Response downloadPdf(
    @PathParam("id") String id) {
    Report report = svc.find(id);
    StreamingOutput stream = out -> {
        pdfGen.generate(report, out);
        out.flush();
    };
    return Response
        .ok(stream)
        .header("Content-Disposition",
            "attachment; filename=report.pdf")
        .build();
}

// StreamingOutput avoids buffering
// — ideal for large binary responses
```

Delivering a File

java.io.File and java.nio.file.Path as response entities — with security

```
@GET @Path("/files/{filename}")
public Response download(
    @PathParam("filename") String filename,
    @QueryParam("inline") @DefaultValue("false") boolean inline)
    throws IOException {

    // — Security: path traversal guard —
    Path file = BASE_DIR.resolve(filename).normalize();
    if (!file.startsWith(BASE_DIR.normalize()))
        return Response.status(403).build(); // FORBIDDEN
    if (!Files.exists(file)) return Response.status(404).build();

    String mime = Files.probeContentType(file);
    if (mime == null) mime = APPLICATION_OCTET_STREAM;
    String cd = (inline ? "inline" : "attachment") +
        "; filename=\"" + file.getFileName() + "\"";
    return Response.ok(file.toFile(), mime)
        .header("Content-Length", Files.size(file))
        .header("Content-Disposition", cd)
        .header("Cache-Control", "private, max-age=3600")
        .build();
}
```

Path Traversal — The Attack and the Fix

GET /files/../../etc/passwd → Without protection this reads the system password file!

```
// WRONG — never resolve user input directly
Path file = Paths.get("/opt/exports/").resolve(filename); // vulnerable!

// CORRECT — always normalise and verify the result is inside the base directory
private static final Path BASE_DIR = Path.of("/opt/app/exports").normalize();

private Path safeResolve(String filename) {
    if (filename.contains("/") || filename.contains("\\"))
        throw new BadRequestException("Filename must not contain separators");

    Path resolved = BASE_DIR.resolve(filename).normalize();
    if (!resolved.startsWith(BASE_DIR)) // escape check
        throw new ForbiddenException("Access denied");
    return resolved;
}
```

Reject filenames with / or \\

Always call .normalize() after .resolve()

Check result .startsWith(BASE_DIR)

Return 403 Forbidden on violation (not 404 — don't leak information)

Key Takeaways

01

JAX-RS converts parameter strings to primitives, boxed types, enums, and types with `fromString()` / `valueOf()`.

02

`@Produces` matches Accept header; `@Consumes` matches Content-Type. Method-level overrides class-level.

03

Seven `@Param` annotations inject data from different parts of the HTTP request. `@BeanParam` aggregates them.

04

`@DefaultValue` fires only when a parameter is absent — not when it's present but blank or empty.

05

Return `void`→204, `POJO`→200, `Response` for full control. Use `GenericEntity` for generic type preservation.

06

`StreamingOutput` writes lazily — prefer it over `byte[]` for large files to avoid memory exhaustion.

07

Always normalise paths and verify against `BASE_DIR` before serving files. Return 403 on traversal attempt.

Thank You

Questions & Lab Time

Lab exercises:

- Lab 1: Simple types — primitives, enums, custom ParamConverter
- Lab 2: @Consumes/@Produces — content negotiation
- Lab 3: @XXXParam — all param sources + @BeanParam
- Lab 4: Return types — Response builder patterns
- Lab 5: Binary + file download + path traversal security

